## Oracle

The default fetch size is set to 10 records, as a consequence of the JDBC driver memory model.

The Oracle 10i and 11g drivers preallocate a `byte[]` and a `char[]` buffers at statement creation time, whose lengths are given by the multiplication of the fetch size by the maximum memory footprint of each selected column. A `VARCHAR2(N)` column can accommodate at most N bytes. Storing a field with a maximum size of 5 characters into a `VARCHAR2(4000)` column would preallocate 4000 bytes on the client-side, which is definitely a waste of memory.

Avoiding memory allocation, by reusing existing buffers, is a very solid reason for employing statement caching. Only when using the implicit statement cache, the 10i and 11g drivers can benefit from recycling client-side memory buffers.

The 12c implementation[a] defers the buffer allocation until the result set is ready for fetching. This driver version uses two `byte[]` arrays instead, which are allocated lazily. Compared to the previous versions, the 12c memory footprint is greatly reduced since, instead of allocating the maximum possible data storage, the driver uses the actually extracted data size.

Although the optimal fetch size is application-specific, being influenced by the data size and the runtime environment concurrency topology, the Oracle JDBC driver specification recommends limiting the fetch size to at most 100 records.

Like with any other performance optimization, these indications are suggestions at best, and measuring application performance is the only viable way of finding the right fetch size.

[a] http://www.oracle.com/technetwork/database/application-development/jdbc-memory-management-12c-1964666.pdf

## SQL Server

The SQL Server JDBC driver uses *adaptive buffering*[a], so the result set is fetched in batches, as needed. The size of a batch is therefore automatically controlled by the driver.

Although enabled by default, adaptive buffering is limited to forward-only and read-only cursors. Both scrollable and updatable result sets operate on a single block of data, whose length is determined by the current statement fetch size.

[a] https://msdn.microsoft.com/en-us/library/bb879937%28v=sql.110%29.aspx

### PostgreSQL

The entire result set is fetched at once[a] into client memory. The default fetch size requires only one database roundtrip, at the price of increasing the driver memory consumption. By changing fetch size, the result set is associated with a database cursor, allowing data to be fetched on demand.

[a] https://jdbc.postgresql.org/documentation/head/query.html

### MySQL

Because of the network protocol design consideration[a], fetching the whole result set is the most efficient data retrieval strategy. The only streaming option requires processing one row at a time, and the number of database roundtrips is given by the read-ahead buffer configuration.

[a] http://dev.mysql.com/doc/connector-j/en/connector-j-reference-implementation-notes.html

The following graph captures the response time of four database systems when fetching 10 000 rows while varying the fetch size of the forward-only and read-only `ResultSet`.

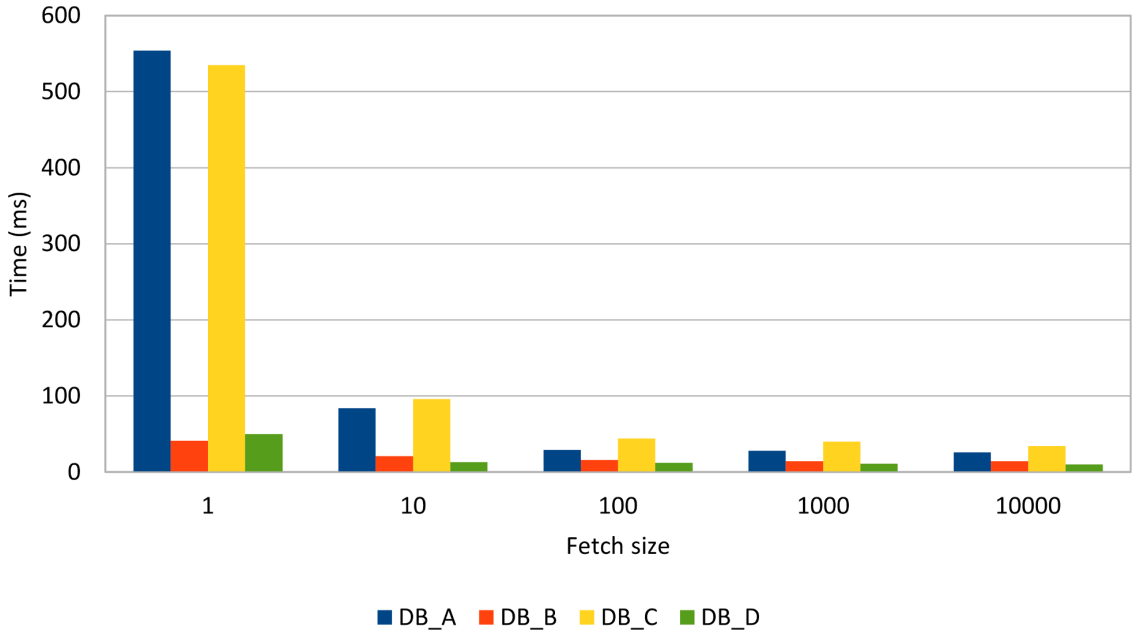


**Figure 5.2: ResultSet fetch size**

Fetching one row at a time requires 10 000 roundtrips, and the networking overhead impacts response time. Up to 100 rows, the fetch size plays an important role in lowering retrieval

time (only 100 roundtrips), but beyond this point, the gain becomes less noticeable.

## 5.5 ResultSet size

Setting the appropriate fetching size can undoubtedly speed up the result set retrieval, as long as a statement fetches only the data required by the current business logic. All too often, unfortunately, especially with the widespread of ORM tools, the statement might select more data than necessary. This issue might be caused by selecting too many rows or too many columns, which are later discarded in the data access or the business layer.

### 5.5.1 Too many rows

Tables tend to grow (especially if the application gains more traction), and, with time, a moderate result set might easily turn into a performance bottleneck. These issues are often discovered in production systems, long after the application code was shipped.

A user interface can accommodate just as much info as the view allows displaying. For this reason, it is inefficient to fetch a whole result set if it cannot fit into the user interface. Pagination or dynamic scrolling are common ways of addressing this issue, and partitioning data sets becomes unavoidable.

Limiting result sets is common in batch processing as well. To avoid long-running transactions (which might put pressure on the database undo/redo logs), and to also benefit from parallel execution, a batch processor divides the current workload into smaller jobs. This way, a batch job can take only a subset of the whole processing data.

When the result set size is limited by external factors, it makes no sense to select more data than necessary.

Without placing upper bounds, the result sets grow proportionally with the underlying table data. A large result set requires more time to be extracted and to be sent over the wire too.

Limiting queries can, therefore, ensure predictable response times and database resource utilization. The shorter the query processing time, the quicker the row-level locks are released, and the more scalable the data access layer becomes.

There are basically two ways of limiting a result set.

The former and the most efficient strategy is to include the row restriction clause in the SQL statement. This way, the Optimizer can better come up with an execution plan that is optimal for the current result set size (like selecting an index scan instead of a table scan).

The latter is to configure a maximum row count at the JDBC `Statement` level. Ideally, the driver can adjust the statement to include the equivalent result set size restriction as a SQL clause, but, most often, it only hints the database engine to use a database cursor instead.

#### 5.5.1.1 SQL limit clause

### SQL:2008

Although the SQL:2008 added support for limiting result sets, only starting from Oracle 12c[a], SQL Server 2012[b] and PostgreSQL 8.4[c], the standard syntax started being supported.

```
SELECT pc.id AS pc_id, p.title AS p_title
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
ORDER BY pc_id
OFFSET ? ROWS
FETCH FIRST (?) ROWS ONLY
```

Surrounding the *row count* placeholder with parentheses is a workaround for a PostgreSQL database issue[d]. On SQL Server it works with or without the enclosing parentheses.

Older database versions or other database systems (e.g. MySQL 5.7) still rely on a vendor-specific syntax to restrict the result set size.

[a] https://docs.oracle.com/database/121/SQLRF/statements_10002.htm#SQLRF01702
[b] https://technet.microsoft.com/en-us/library/gg699618%28v=sql.110%29.aspx
[c] http://www.postgresql.org/docs/current/static/sql-select.html#SQL-LIMIT

### Oracle

Unlike other relational databases, Oracle does not have a reserved keyword for restricting a query result set, but because each record is attributed a result set entry order number (given by the `ROWNUM` virtual column), the syntax for limiting a result set becomes:

```
SELECT *
FROM (
        SELECT pc.id AS pc_id, p.title AS p_title
        FROM post_comment pc
        INNER JOIN post p ON p.id = pc.post_id
        ORDER BY pc_id
)
WHERE ROWNUM <= ?
```

### SQL Server

The `TOP` keyword has been the *de facto* way of restricting the result set size:

```
SELECT TOP (?) pc.id AS pc_id, p.title AS p_title
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
ORDER BY pc_id
```

### PostgreSQL and MySQL

The `LIMIT` keyword places an upper bound on the result set size:

```
SELECT pc.id AS pc_id, p.title AS p_title
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
ORDER BY pc_id
LIMIT ?
```

#### 5.5.1.2 JDBC max rows

The JDBC specification defines the maxRows[1] attribute which limits all `ResultSet(s)` for the current statement.

```
statement.setMaxRows(maxRows);
```

Unlike the SQL construct, the JDBC alternative is portable across all driver implementations. This can be very handy especially when the application needs to support multiple database systems.

According to the JDBC documentation, the driver is expected to discard the extra rows when the maximum threshold is reached.

From a data access performance perspective, dropping extra rows is a poor strategy because it wastes both database resources (CPU, I/O, Memory) as well as networking bandwidth.

[1] http://docs.oracle.com/javase/8/docs/api/java/sql/Statement.html#setMaxRows-int-

## Oracle

When a `ResultSet` is being traversed, the client-side cursor fetches data in chunks (the fetch size attribute controlling the number of records in a chunk).

After each new batch retrieval, the total number of records is checked against the `maxRows` upper bound, and if the threshold is reached, the driver closes the networking stream.

The `maxRows` upper bound can, therefore, prevent the database and the client-side driver from wasting resources on fetching records the client does not even need. However, if the `maxRows` value is small, the Optimizer will not use indexes if the size of the scanned data set is rather large.

## SQL Server

When the `Statement.setMaxRows(int maxRows)`[a] method is called, the driver calls the `SET ROWCOUNT` SQL command:

```
SET ROWCOUNT N
```

Unlike the `TOP` or `FETCH` SQL directives, the `ROWCOUNT` command is taken into consideration only during the execution phase, and it does not influence the plan generation. Because of this, the execution plan might not be optimized for the given result set size, so a table scan might be chosen over an index.

The SQL Server documentation[b] recommends using the SQL directives over the `SET ROWCOUNT` command.

[a] https://msdn.microsoft.com/en-us/library/ms378838%28v=sql.110%29.aspx

## PostgreSQL

The JDBC driver takes the `maxRows` statement attribute and sends it along with the query being executed. With this info, the Optimizer can choose an execution plan that is tailored for the given result set size, and it might even avoid some expensive operations like sorting the whole projection. The Extractor can also close the database cursor right after it fetched the desired number of records, therefore sparing both database and networking resources.

> **MySQL**
>
> The `maxRows` attribute is not sent to the database server, so neither the Optimizer nor the Extractor can benefit from this hint. While the JDBC driver would normally fetch all rows, by placing an upper bound on the result set size, the client-side can spare some networking overhead.

#### 5.5.1.3 Less is more

The following test is going to demonstrate the performance improvement of limiting the result set size. The test data set consists of 100 000 *post* and 1 000 000 *comment* entries. In the first round, the entire result set is being fetched, and the response time is going to be proportional to the projection size. By limiting the result set to 100 records, either by using SQL or the JDBC `maxRows` setting, the response time is going to drop significantly.

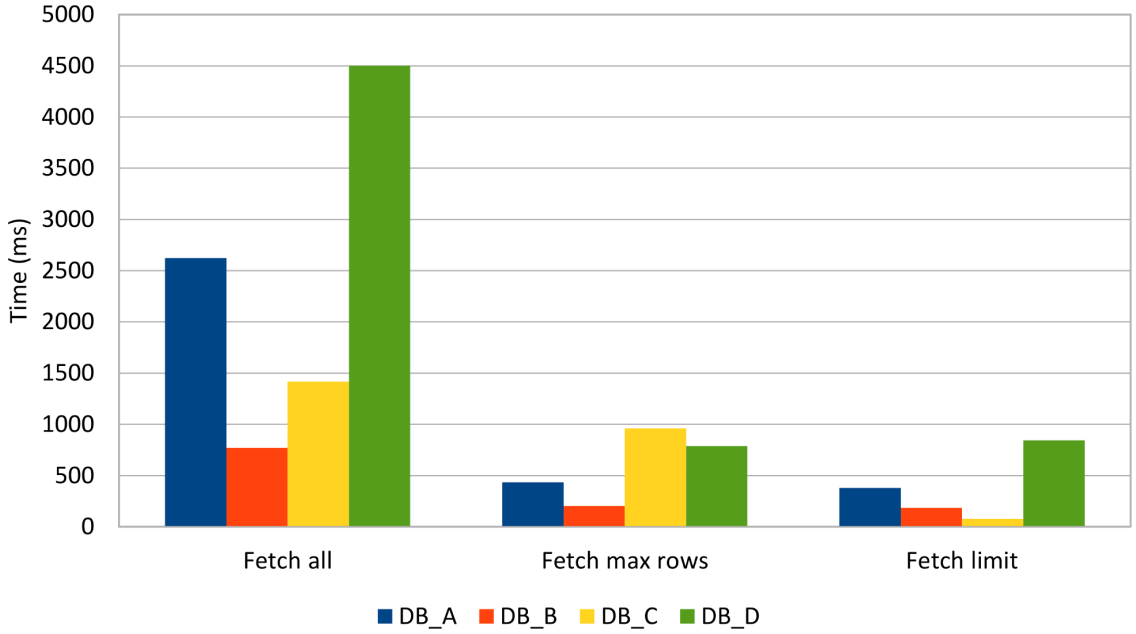


**Figure 5.3: ResultSet size**

The test results confirm the previous assumptions and the SQL level restriction proves to be the optimal strategy for limiting a result set. The `maxRows` driver implementation yields a surprisingly good result, especially when taking into consideration the JDBC specification on dropping extra records. Fetching a large result set puts much pressure on database resources, which does not only affect the current processing unit of work. Other concurrent transactions can also exhibit longer processing times, as a consequence of database resources shortage.

### 5.5.2 Too many columns

Not only fetching too many rows can cause performance issues but even extracting too many columns can increase the result set processing response time. The next test case is going to select 100 *posts* with *details* and their associated 1000 *comments*, using one of following two statements:

```
SELECT *
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
INNER JOIN post_details pd ON p.id = pd.id

SELECT pc.version
FROM post_comment pc
INNER JOIN post p ON p.id = pc.post_id
INNER JOIN post_details pd ON p.id = pd.id
```

The following graph depicts the execution times of fetching all columns, as opposed to extracting only a subset of the whole column projection.

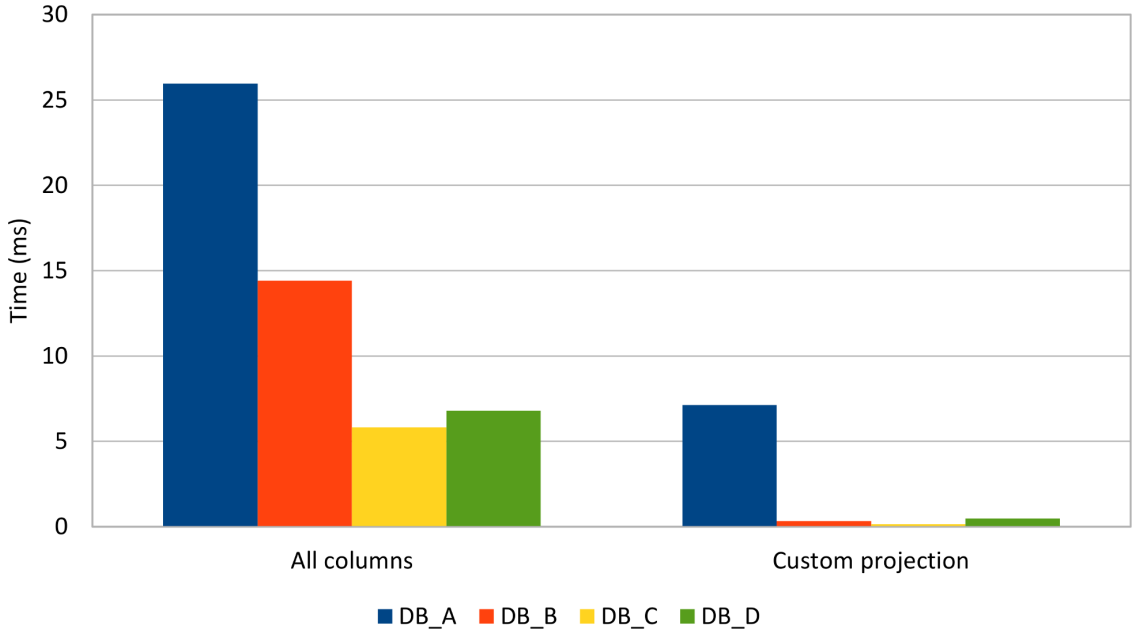


**Figure 5.4: ResultSet projection size**

This situation is more prevalent among ORM tools, as for populating entities entirely, all columns are needed to be selected. This might pass unnoticed when selecting just a few entities, but, for large result sets, this can turn into a noticeable performance issue.

If a business case requires only a subset of all entity attributes, fetching extra columns will become a waste of database and application resources (CPU, Memory, I/O, Networking).

# 6. Transactions

A database system must allow concurrent access to the underlying data. However, shared data means that read and write operations must be synchronized to ensure that data integrity is not compromised.

To control concurrent modifications, the Java programming language defines the `synchronized` keyword for two purposes:

- It can restrict access to a shared `Object` (to preserve invariants), so only a `Thread` can execute a routine at any given time.
- It propagates changes from the current `Thread` local memory to the global memory that is available to all running threads of executions.

This behavior is typical for other concurrent programming environments and database systems are no different. In a relational database, the mechanism for ensuring data integrity is implemented on top of transactions.

A transaction is a collection of read and write operations that can either succeed or fail together, as a unit. All database statements must execute within a transactional context, even when the database client does not explicitly define its boundaries.

In 1981, Jim Gray first defined the properties of a database transaction in his famous paper: The transaction concept: virtues and limitations[1]. Both this paper and the first versions of the SQL standard (SQL-86 and SQL-89) only used three properties for defining a database transaction: *Atomicity*, *Consistency*, and *Durability*.

Along with other relation database topics, the transaction research has continued ever since, and so the SQL-92 version introduced the concept of *Isolation Levels*. These four properties have been assembled in the well-known *ACID* (*Atomicity*, *Consistency*, *Isolation*, *and Durability*) acronym that soon became synonym with relation database transactions.

Knowing how database transactions work is very important for two main reasons:

- effective data access (data integrity should not be compromised when aiming for high-performance)
- efficient data access (reducing contention can minimize transaction response time which, in turn, increases throughput).

The next sections detail each transaction property in relation to high-performance data processing.

[1] http://research.microsoft.com/en-us/um/people/gray/papers/theTransactionConcept.pdf

## 6.1 Atomicity

Atomicity is the property of grouping multiple operations into an all-or-nothing unit of work, which can succeed only if all individual operations succeed. For this reason, the database must be able to roll back all actions associated with every executed statement.

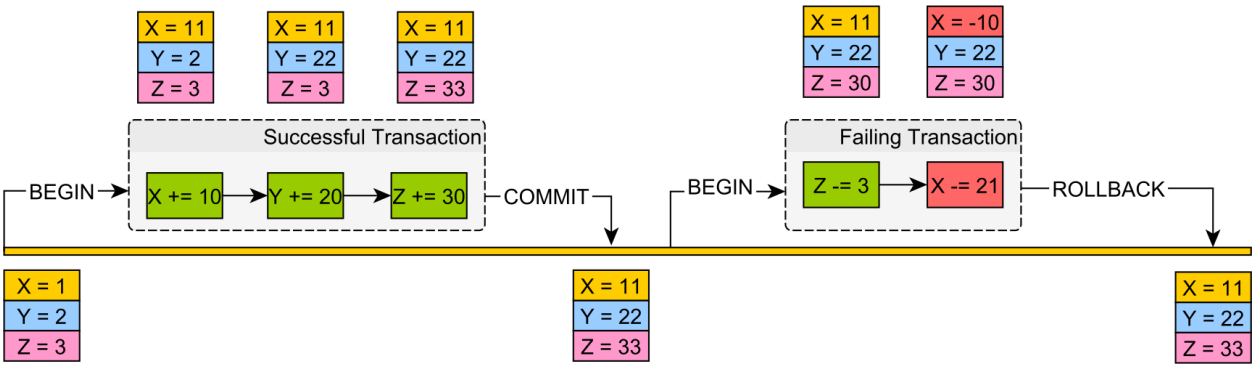


**Figure 6.1: Atomic units of work**

### Write-write conflicts

Ideally, every transaction would have a completely isolated branch which could be easily discarded in case of a rollback. This scenario would be similar to how a *Version Control System* (e.g. *git*) implements branching. In case of conflicts, the Virtual Control System aborts the commit operation, and the client has to manually resolve the conflict. Unlike VCS tools, the relational database engine must manage conflicts without any human intervention.

For this reason, the database prevents write-write conflict situations, and only one transaction can write a record at any given time.

All statements are executed against the actual data structures (tables, indexes, in-memory buffers), only to be materialized at commit time. In case of rollback, the database must revert any pending changed datum to its previous state.

### Oracle

The *undo tablespace*[a] stores the previous data versions in *undo segments*. Upon rolling back, the database engine searches the associated *undo segments* that can recreate the *before image* of every datum that was changed by the currently running transaction.

[a] https://docs.oracle.com/database/121/ADMIN/undo.htm#ADMIN11460

## SQL Server

The *transaction log*[a] stores details about the currently running transactions and their associated modifications. The rollback process scans the *transaction log* backward to find the associated undo records. When the record is found, the database engine restores the before image of the affected datum.

To prevent the transaction log from filling up, the log must be truncated on a regular basis. Long-running transactions can delay the truncation process, so that is another reason to avoid them as much as possible.

[a] https://msdn.microsoft.com/en-us/library/ms190925.aspx

## PostgreSQL

Unlike other database systems, PostgreSQL does not use a dedicated append-only undo log. Because of its multi-version nature, every database object maintains its own version history. In the absence of the log seek-up phase, the rollback process becomes much lighter as it only requires to switch from one version to the other.

The downside is that the previous version space is limited in size, and so it must be reused. The process of reclaiming the storage occupied by old versions is called VACUUMING.

Each transaction has an associated XID[a], and newer transactions must have a greater XID number than all previous ones.

The transaction XID is a 32-bit number so it can accommodate over 4 billion transactions. In a high-performance application, the transaction lifespan is very short, and if the VACUUM process is disabled, this threshold may be reached. When the XID counter reaches its maximum value, it wraps around and start again from zero.

The transactions issued prior to the XID wraparound have their identifiers greater than newer transactions started after the XID counter reset. This anomaly can cause the system to perceive older transactions as they were started in the future, which can lead to very serious data integrity issues.

[a] http://www.postgresql.org/docs/current/static/routine-vacuuming.html

### MySQL

The *undo log* is stored in the *rollback segment*[a] of the system tablespace.

Each *undo log* is split into two sections, one responsible for rolling back purposes and the other for reconstructing the before image. The first section can be wiped out right after the transaction is ended, while the other needs to linger for as long as any currently running query or other concurrent transactions need to see a previous version of the records in question.

Behind the scenes, MySQL runs a *purge* process that cleans up the storage occupied by deleted records, and it also reclaims the undo log segments that are no longer required.

Long-running transactions delay the *purge* process execution, causing the undo log to grow very large, especially in write-heavy data access scenarios.

[a] https://dev.mysql.com/doc/refman/5.7/en/innodb-multi-versioning.html

## 6.2 Consistency

A modifying transaction can be seen as a state transformation, moving the database from one valid state to another. The relational database schema ensures that all primary modifications (insert/update/delete statements), as well as secondary ones (issued by triggers), obey certain rules on the underlying data structures:

- column types
- column length
- column nullability
- foreign key constraints
- unique key constraints
- custom *check* constraints.

Consistency is about validating the transaction state change so that all committed transactions leave the database in a proper state. If only one constraint gets violated, the entire transaction will be rolled back, and all modifications are going to be reverted.

Although the application must validate user input prior to crafting database statements, the application-level checks cannot span over other concurrent requests, possibly coming from different web servers. When the database is the primary integration point, the advantages of a strict schema become even more apparent.

## MySQL

Traditionally, MySQL constraints are not strictly enforced[a], and the database engine replaces invalid values with predefined defaults:

- Out-of-range numeric values are set to either 0 or the maximum possible value.
- `String` values are trimmed to the maximum length.
- Incorrect Date/Time values are permitted (e.g. 2015-02-30).
- NOT NULL constraints are only enforced for single INSERT statements. For multi-row inserts, 0 replaces a null numeric value, and `''` is used for a null `String`.

Since the 5.0.2 version, *strict* constraints are possible if the database engine is configured to use a custom *sql mode*[b]:

```
SET GLOBAL sql_mode='POSTGRESQL,STRICT_ALL_TABLES';
```

Because the `sql_mode` resets on server startup, it is better to set it up in the MySQL configuration file:

```
[mysqld]
sql_mode = POSTGRESQL,STRICT_ALL_TABLES
```

[a] https://dev.mysql.com/doc/refman/5.7/en/constraint-invalid-data.html
[b] https://dev.mysql.com/doc/refman/5.7/en/sql-mode.html

## Consistency as in CAP Theorem

According to the CAP theorem[a], when a distributed system encounters a network partition, the system must choose either *Consistency* (all changes are instantaneously applied to all nodes) or *Availability* (any node can accept a request), but not both. While in the definition of ACID, consistency is about obeying constraints, in the CAP theorem context, consistency refers to *linearizability*[b], which is an isolation guarantee instead.

[a] https://en.wikipedia.org/wiki/CAP_theorem
[b] http://www.bailis.org/blog/linearizability-versus-serializability/

## 6.3 Isolation

If there were only one user accessing the database, there would not be any risk of data conflicts. According to the Universal Scalability Law, if the sequential fraction of the data access patterns is less than 100%, the database system may benefit from parallelization.

By offering multiple concurrent connections, the transaction throughput can increase, and the database system can accommodate more traffic. However, parallelization imposes additional challenges as the database must interleave transactions in such a way that conflicts do not compromise data integrity. The execution order of all the currently running transaction operations is said to be *serializable* when its outcome is the same as if the underlying transactions were executed one after the other.

The serializable execution is, therefore, the only transaction isolation level that does not compromise data integrity while allowing a certain degree of parallelization. In 1981, Jim Gray described the largest airlines and banks as having 10 000 terminals and 100 active transactions, which explains why, up until SQL-92, serializable was the *de facto* transaction isolation level.

### 6.3.1 Concurrency control

To manage data conflicts, several concurrency control mechanisms have been developed throughout the years. There are two strategies for handling data collisions:

- Avoiding conflicts (e.g. *two-phase locking*) requires locking to control access to shared resources.
- Detecting conflicts (e.g. *Multi-Version Concurrency Control*) provides better concurrency, at the price of relaxing serializability and possibly accepting various data anomalies.

#### 6.3.1.1 Two-phase locking

In 1976, Kapali Eswaran and Jim Gray (et al.) published The Notions of Consistency and Predicate Locks in a Database System[2] paper, which demonstrated that serializability could be obtained if all transactions used the *two-phase locking* (*2PL*) protocol.

Initially, all database systems employed 2PL for implementing serializable transactions, but, with time, many vendors have moved towards an MVCC (Multi-Version Concurrency Control) architecture. By default, SQL Server still uses locking for implementing the Serializability isolation level.

Because 2PL guarantees transaction serializability, it is very important to understand the price of maintaining strict data integrity on the overall application scalability and transaction performance.

[2] http://research.microsoft.com/en-us/um/people/gray/papers/On%20the%20Notions%20of%20Consistency%20and%20Predicate%20Locks%20in%20a%20Database%20System%20CACM.pdf

However, locking is not used only in 2PL implementations, and, to address both DML and DDL statement interaction and to minimize contention on shared resources, relational database systems use Multiple granularity locking[3].

Database objects are hierarchical in nature, a logical tablespace being mapped to multiple database files, which are built of data pages, each page containing multiple rows. For this reason, locks can be acquired on different database object types.

Locking on lower levels (e.g. rows) can offer better concurrency as it reduces the likelihood of contention. Because each lock takes resources, holding multiple lower-level locks can add up, so the database might decide to substitute multiple lower-level locks into a single upper-level one. This process is called *lock escalation*, and it trades off concurrency for database resources.

Each database system comes with its own lock hierarchy, but the most common types (even mentioned by the 2PL initial paper) remain the following ones:

- shared (read) lock, preventing a record from being written while allowing reads
- exclusive (write) lock, disallowing both read and write operations.

Locks alone are not sufficient for preventing conflicts. A concurrency control strategy must define how locks are being acquired and released because this also has an impact on transaction interleaving.

For this purpose, the 2PL protocol defines a lock management strategy for ensuring serializability. The 2PL protocol splits a transaction into two sections:

- expanding phase (locks are acquired, and no lock is released)
- shrinking phase (all locks are released, and no other lock is further acquired).

In lock-based concurrency control, all transactions must follow the 2PL protocol, as otherwise serializability might be compromised, resulting in data anomalies.

### Transaction schedule

To provide recovery from failures, the transaction schedule (the sequence of all interleaved operations) must be strict. If a write operation, in a first transaction, happens-before a conflict occurring in a subsequent transaction, in order to achieve transaction *strictness*, the first transaction commit event must also happen before the conflict.

Because operations are properly ordered, strictness can prevent *cascading aborts* (*one transaction rollback triggering a chain of other transaction aborts, to preserve data consistency*). Releasing all locks only after the transaction has ended (either commit or rollback) is a requirement for having a strict schedule.

[3] https://en.wikipedia.org/wiki/Multiple_granularity_locking

The following diagram shows how transaction interleaving is coordinated by 2PL:

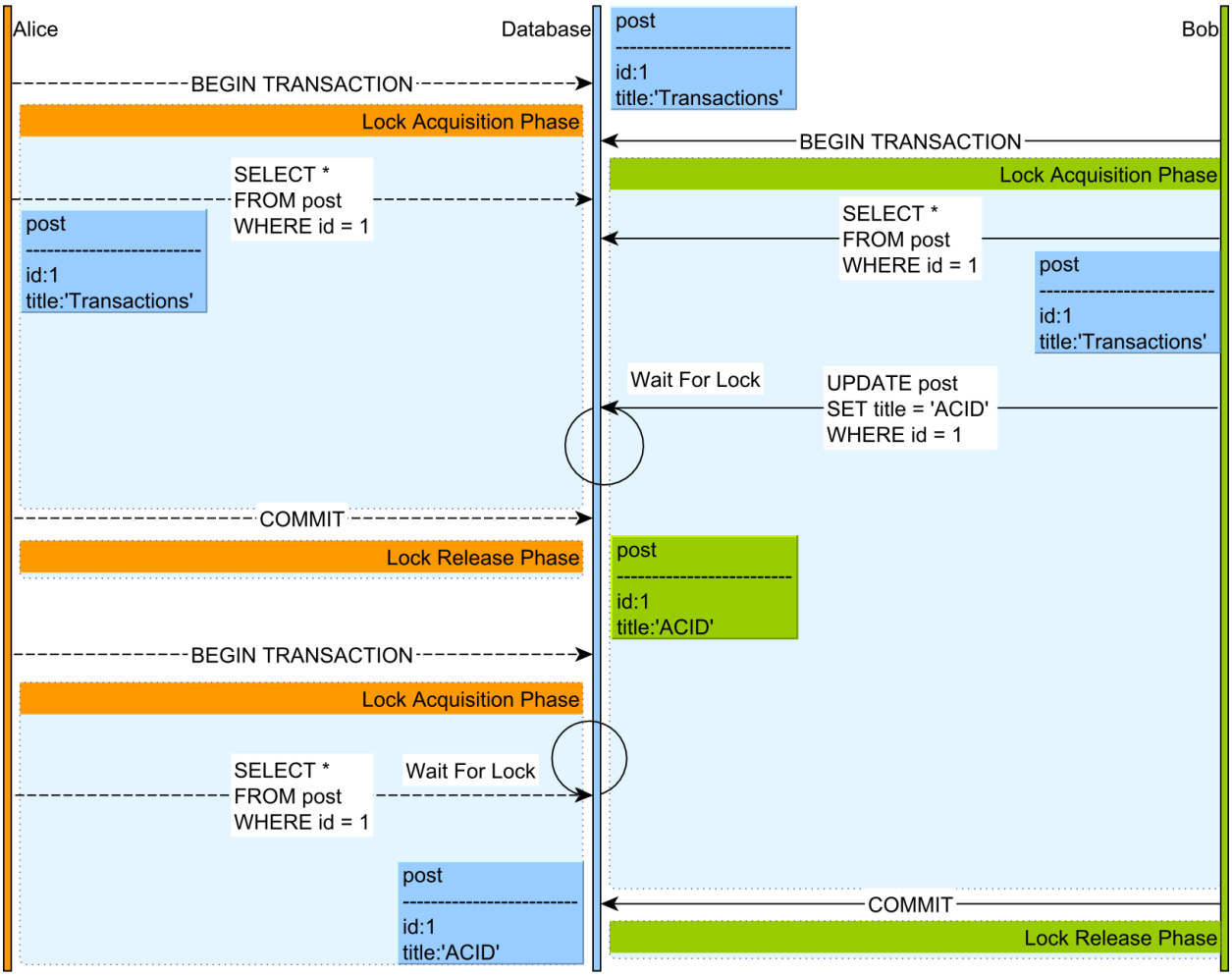


**Figure 6.2: Two-phase locking**

- Both *Alice* and *Bob* select a *post* record, both acquiring a shared lock on this record.
- When *Bob* attempts to update the *post* entry, his statement is blocked by the Lock Manager because *Alice* is still holding a shared lock on this database row.
- Only after *Alice*'s transaction ends and all locks are being released, *Bob* can resume his update operation.
- *Bob*'s update generates a lock upgrade, so the shared lock is replaced by an exclusive lock, which prevents any other concurrent read or write operation.
- *Alice* starts a new transaction and issues a select query for the same *post* entry, but the statement is blocked by the Lock Manager since *Bob* owns an exclusive lock on this record.
- After *Bob*'s transaction is committed, all locks are released, and *Alice*'s query can be resumed, so she gets the latest value of this database record.

## Deadlocks

Using locking for controlling access to shared resources is prone to deadlocks, and the transaction scheduler alone cannot prevent their occurrences.

A deadlock happens when two concurrent transactions cannot make progress because each one waits for the other to release a lock. Because both transactions are in the lock acquisition phase, neither one releases a lock prior to acquiring the next one.

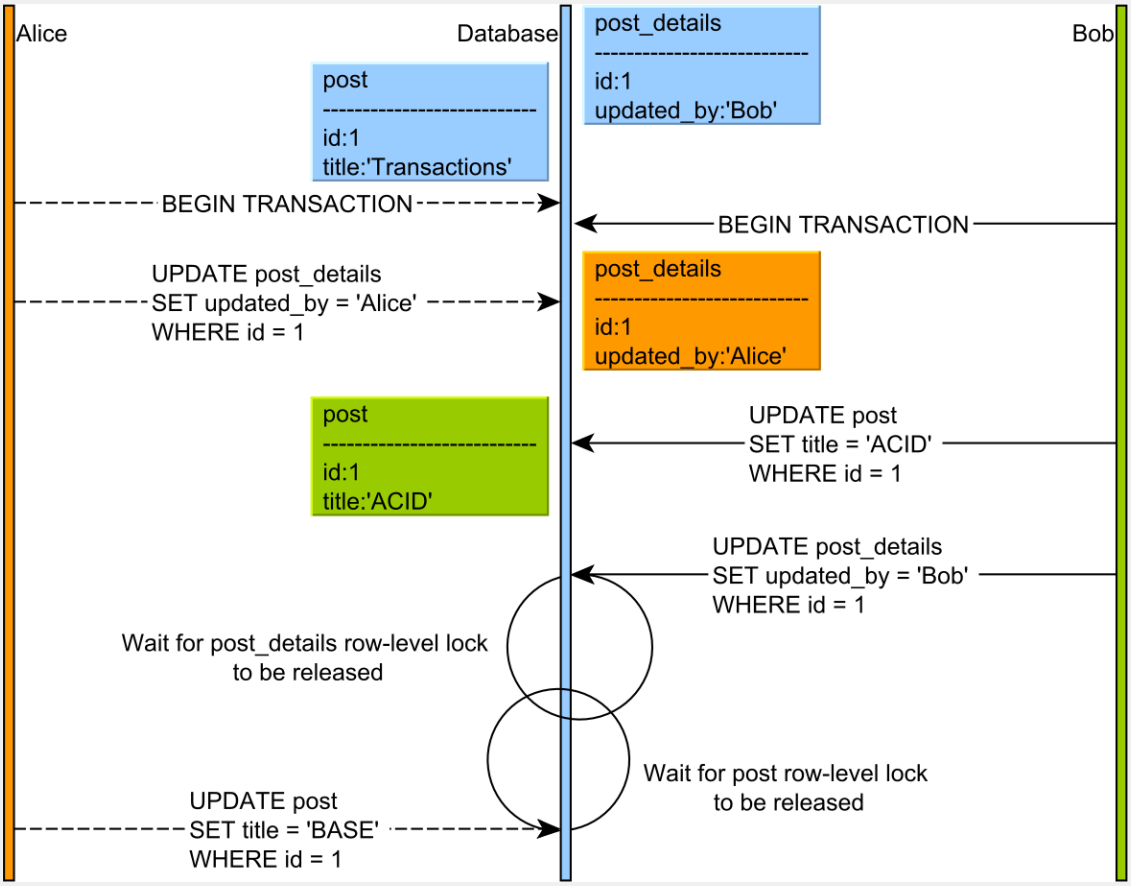


**Figure 6.3: Dead lock**

Preserving the lock order becomes the responsibility of the data access layer, and the database can only assist in recovering from a deadlock situation.

The database engine runs a separate process that scans the current conflict graph for lock-wait cycles (which are caused by deadlocks). When a cycle is detected, the database engine picks one transaction and aborts it, causing its locks to be released, so the other transaction can make progress.

#### 6.3.1.2 Multi-Version Concurrency Control

Although locking can provide a serializable transaction schedule, the cost of lock contention can undermine both transaction response time and scalability. The response time can increase because transactions must wait for locks to be released, and long-running transactions can slow down the progress of other concurrent transactions as well. According to both Amdahl's Law and the Universal Scalability Law, concurrency is also affected by contention.

To address these shortcomings, the database vendors have opted for optimistic concurrency control mechanisms. If 2PL prevents conflicts, Multi-Version Concurrency Control (MVCC) uses a conflict detection strategy instead.

The promise of MVCC is that readers do not block writers and writers do not block readers. The only source of contention comes from writers blocking other concurrent writers, which otherwise would compromise transaction rollback and atomicity.

To prevent blocking, the database can rebuild previous versions of a database record so an uncommitted change can be hidden away from incoming concurrent readers. The lack of locking makes it more difficult to implement a serializable schedule, so the database engine must analyze the current interleaving operations and detect anomalies that would compromise serializability.

### Oracle

Oracle does not implement 2PL at all, relying on MVCC mechanism for managing concurrent data access. Every query gets a point-in-time data snapshot and, depending on the isolation level, the timestamp reference can be relative to the current statement or to the current transaction start time.

To rebuild previous record versions, Oracle uses the undo segments[a], which already contain all the necessary data required for rolling back an uncommitted change. The point-in-time is based on the System Change Number (SCN), which is a logical timestamp reference and, unlike physical time, is guaranteed to be incremented monotonically.

Apart from MVCC, Oracle also supports explicit locking as well, using the `SELECT FOR UPDATE` SQL syntax.

[a] https://docs.oracle.com/database/121/CNCPT/consist.htm#CNCPT221

## SQL Server

By default, SQL Server uses locks for implementing all the isolation levels stipulated by the SQL standard.

For the *Read Committed* isolation level to take advantage of the MVCC model, the following configuration must be set first:

```
ALTER DATABASE high_performance_java_persistence
SET READ_COMMITTED_SNAPSHOT ON;
```

For a higher-level isolation, SQL Server offers the *Snapshot* isolation mode, which must be activated at the database level:

```
ALTER DATABASE high_performance_java_persistence
SET ALLOW_SNAPSHOT_ISOLATION ON;
```

Because Snapshot is a custom isolation level, it must also be set at the connection level prior to starting a new transaction:

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
GO
BEGIN TRANSACTION;
GO
COMMIT TRANSACTION;
GO
```

After enabling row versioning, the database can track record changes in the *tempdb* database.

When a row is either updated or deleted, the current row entry holds a reference back to the previous version, which is recorded in the version store, in the tempdb database. Rows are not deleted right away but only marked for deletion, the actual removal being done by the *Ghost cleanup task*. Old versions must be kept for as long as a currently running transaction might need them, which is specified by the transaction isolation level.

The Ghost cleanup task runs periodically and reclaims storage from old versions that are no longer necessary.

A long-running transaction would require the database engine to keep some old version for a very long time, and, because version changes are chained in a linked list structure, restoring a very old version might becomes resource intensive.

## PostgreSQL

Unlike all other database systems, PostgreSQL stores both the current rows and their previous versions (even the ones for the aborted transactions) in the actual database table. Like Oracle, PostgreSQL embraces the MVCC data access model, and it does not offer a 2PL transaction isolation implementation at all.

Each table row has two additional columns (*xmin* and *xmax*), which are used to control the visibility of various row versions. When a row is inserted, the current transaction identifier is stored in the xmin column.

Both the update and the delete operations end up creating a new row entry with a xmax column storing the current transaction identifier.

The *Vacuum* cleaner process runs regularly and reclaims storage occupied by deleted entries (and successfully committed) or by previous versions that are no longer required by the currently running transactions.

Although PostgreSQL is seen as a pure MVCC model, locking is still required to prevent write-write conflicts or for explicit locking[a]. `SELECT FOR UPDATE` is used to acquire an exclusive row-level lock, while `SELECT FOR SHARE` is for applying a shared lock instead.

[a] http://www.postgresql.org/docs/current/static/explicit-locking.html

## MySQL

The InnoDB storage engine offers support for ACID transactions and uses MVCC for controlling access to shared resources. The InnoDB MVCC implementation is very similar to Oracle, and previous versions of database rows are stored in the rollback segment as well.

When a transaction demands a previous row version, MySQL must reconstruct it from rollback segments. Delete operations just mark an entry as being ready for deletion, and the *purge* thread is going to do the actual physical cleanup.

Both the transaction rollback and the previous row version restoring processes (required by a given transaction visibility guarantees) are very much the same thing.

Like other database systems, MySQL also offers explicit locking[a] for when MVCC is no longer satisfactory. A shared lock is acquired using `SELECT LOCK IN SHARE MODE`, while, for exclusive locks, the much more common `SELECT FOR UPDATE` syntax is being used.

[a] https://dev.mysql.com/doc/refman/5.7/en/innodb-locking-reads.html